Product Pitch Scripts

产品介绍稿 · 18 个项目 · 每个约 2 分钟 · 打印审阅版

ⓘ 怎么读 · How to read

- 每个项目两页：正面是八拍讲稿，背面是关键技术深挖（双面打印正好一张纸）。
  Two pages per project: the pitch on the front, the technical deep dive on the back.
- **粗体中文**是要点提示；下面是要念的英文。
  Bold Chinese is the cue; the English below is what you say.
- **黄色 = 关键技术**：面试官会追问，要能展开。
  Yellow = key tech you should be able to expand on.
- **红色 = 取舍**：故意放弃了什么、为什么。
  Red = a deliberate trade-off and why.
- 右边留白用来写修改意见。数字都来自作品页上带出处的指标或你的 Vault 笔记。
  Use the right margin for edits. Numbers come from the sourced portfolio metrics or your Vault notes.

ⓘⓘ 目录 · Contents

01 EagleFrontier · 投资平台工程 · Investment platform

02 IPOT · 餐厅预订、收银与 Nalu · Restaurant platform

03 Governance · AI 使用治理平台 · AI governance platform

04 PII Detector · 浏览器内隐私检测 · In-browser PII detector

05 Refund Agent · 退款智能体 · Refund agent

06 Dispute Ingest · 信用卡争议接入 · Card dispute ingestion

07 Loan Draw Review · 放款审核控制 · Loan-draw review controls

08 Career OPS · 求职数据与面试练习 · Job market feed

09 Remote Commander Ops · 远程控制守护 · Remote access supervision

10 双语论文朗读器 · Bilingual Note Reader

11 REEA · AI 房产平台 · AI real-estate platform

12 LightUp · 看得见影响的捐赠 · Giving that shows impact

13 Polygraf.ai 实习 · PII 脱敏与治理后台 · Polygraf internship

14 开放投融资数据 · Open Private Markets

15 AI Infrastructure Commander · 自建 MCP 工具服务器（规划中）· Self-hosted MCP tool server

16 硕士论文重做 · Revisiting my MSc thesis

17 学习视频生成 · Narrated Study Videos

18 LLM 架构卡片 · LLM Arch Cards

EagleFrontier · 投资平台工程 · Investment platform

1 谁用、什么问题 · Who and why

旧金山商人银行，服务家族办公室和投资人；流程靠人工和 HubSpot，尽调数据受保密协议约束，不能交给云端 AI

EagleFrontier is a San Francisco merchant bank. It works with family offices and investors on private-market deals: sourcing companies, M&A advisory, and diligence. That work ran on manual research and HubSpot, and diligence data is under NDA, so it can't go to a cloud AI.

2 它是什么 · What it is

一个登录后的平台：说要找什么公司 → 智能体搜和筛 → 人工审核 → 客户门户和 CRM → 尽调问答

So we built one platform behind a login. A client says what companies it's looking for, agents find and screen candidates, our team reviews them, and approved ones land in that client's portal, with a CRM for companies and deals. Investors can then ask diligence questions about them.

3 怎么运作 · How it works

门户在云上，只查批准、转发、不存；回答在自己的机器上，数字由代码算，带出处

The portal runs in the cloud (FastAPI, Postgres, Next.js). For diligence it only **checks approval and relays the question over a private network**, and it **stores no questions or answers**. A service on **a machine we own** answers from approved data, with **totals and margins computed by code**, and every answer cites its source file and row.

4 三个保证 · What you can count on

原始数据不出机器 · 每个数字都核对 · 两步登录、书面批准、客户之间数据分开

That gives three guarantees. **Raw diligence data never leaves our machine.** **Every number in an answer is checked against a cited row or a computed total**, or the answer is held back. And access is tight: **two-step login, written approval that expires after 90 days, and each client sees only its own data**.

5 故意放弃的 · Trade-offs

机器停了就停，不退回云端；数字不交给模型；迁移没到位就不上线

We gave up some things on purpose. **If our machine is down, diligence says so instead of falling back to a cloud model**, because one fallback is one leak. **Numbers come from code, not a language model**, so a missing figure gets “not in the data” instead of a guess. And **a deploy stops before restarting if the database migrations aren’t current**: a slower release, but nothing breaks silently.

6 适合谁 · When to use it

适合要对每个数字负责的保密数据；数据公开的话云端工具更快更便宜

This design fits a firm with confidential data that has to stand behind every number. **With public data, a cloud tool would be faster and cheaper.**

7 我做了什么、现在到哪 · My part and status

我负责平台的生产工程：修部署和上线工具、登录安全、尽调整条链、客户隔离；找公司的智能体和正式的本地服务是同事的

I owned the platform's production engineering: fixing deploys that failed five times behind a green health check, plus a release kit with pre-checks, snapshots and rollback (shipped Oct 28); MFA limits and session handling; the diligence chain end to end, from the approval gate and relay to the code that computes figures and checks every number, which the real local service shares; and per-client isolation. Teammates built the sourcing agents and run that local service. The deploy fix is live, login security is merged, diligence is merged but switched off, and isolation is in review.

8 一句话收尾 · In one line

一个平台从找公司到尽调；保密数据留在自己机器上，每个数字可核对

In one line: one platform from sourcing to diligence, where confidential data stays on our machine and every number can be checked.

关键技术深挖 · Technical deep dive

EagleFrontier · 投资平台工程 · Investment platform

只转发、不存 · A relay that stores nothing

怎么做 云端后端删掉了上传、存储和 6 张尽调表；**删表迁移遇到表里有数据就拒绝执行**。问题只在内存里经过，审计行只记谁、何时、结果、耗时。

为什么 · 取舍 存下来就要保护、要删、要解释；**不存，就没有可泄露的副本**。代价是答错了没法从日志重放原话。

追问 Q *What happens if the relay crashes in the middle of a question?*

答 A The audit row is written **before** the question is forwarded, marked “relaying.” If the process dies, the row stays in that state, so we can see an unfinished request, but no question or answer was ever stored. The client just asks again.

目的地白名单只认 IP · Allow-list by IP literal

怎么做 转发目标只能是 **Tailscale 私网地址（100.64.0.0/10）或本机回环**，写成 IP 字面量；任何主机名、169.254 元数据地址、普通内网段都在 DNS 查询和连接之前就拒绝；缺 token 返回 503。

为什么 · 取舍 主机名可以被改指向别处（DNS 重绑定），一不小心就打到云厂商的元数据服务。**只认 IP 少了灵活性**，换地址要改配置。

追问 Q *Why not allow a hostname for the local service?*

答 A A hostname can be re-pointed. With **DNS rebinding**, a name that looked private could resolve to the cloud metadata address. Checking an IP literal before any DNS lookup removes that path completely.

限流，且失败即关 · Rate limits that fail closed

怎么做 每人每分钟 20、每天 200，**Redis 固定窗口**：INCR 和 EXPIRE NX 放在同一个事务里；每个进程最多 8 个同时转发。**Redis 出错就拒绝请求**，不放行。

为什么 · 取舍 本地只有一台机器，护住它比让每个请求都成功重要。**Redis 故障时问答会不可用**，这是故意的。

追问 Q *Why refuse requests when Redis is down instead of letting them through?*

答 A Because the limit protects a single local machine and the confidential data behind it. **Failing open** would mean an outage of the limiter becomes unlimited access. Failing closed means a short outage of the feature, which is the safer failure.

数字由代码算，答完逐个核对 · Code computes, then every number is checked

怎么做 合计、排名、毛利率由代码算；**发出回答前，答案里每个数字都必须出现在引用的原文行或代码算出的结果里**，对不上就换成「not in the data」。

为什么 · 取舍 语言模型会把数字算错或编出来；**宁可拒答也不答错**。代价是开放式分析不支持。

追问 Q *How do you stop the system from making up a number?*

答 A Numbers never come from the model. Every number in the reply has to match a cited row or a figure our code computed; if any number doesn't, the whole answer is replaced with “not in the data.”

IPOT · 餐厅预订、收银与 Nalu · Restaurant platform

1 谁用、什么问题 · Who and why

一家连锁餐厅需要不怕重试、不怕重复点击、不会被模型乱猜的预订和收银

IPOT is a growing restaurant group. It needed bookings, payments and guest messaging that don't break when a request is retried, a button is tapped twice, or an AI model guesses.

2 它是什么 · What it is

三样东西：线上预订服务、收银的钱路、双语预订助手 Nalu

I built three parts: **a reservation service real guests use**, the POS money path, and Nalu, a bilingual booking assistant where **the model never decides what gets booked**.

3 怎么运作 · How it works

规则读字段 → 含糊就追问 → 明确说「是」才下单 → 幂等键防重复；结账是一个事务

Say a guest writes “table for 4 tomorrow at 7.” **Rules, not the model, read the date, time, party size and name**; anything ambiguous becomes a question. The booking runs only after **a plain, explicit yes**, and it's sent with an **Idempotency-Key**, so a retry replays the same booking instead of making a second one. At the table, settling a bill is **one database transaction**: payment, tip, close and table release together.

4 三个保证 · What you can count on

不会重复订、重复收钱 · 不编造（留出集 33/39 全对、0 编造）· 预订服务已上线

Three things you can count on. A retry or double tap can't double-book or double-charge. Nalu doesn't invent details: on a held-out set, **33 of 39 conversations were fully right with 0 invented values**; the model-only pilot got 17 of 30 on a different set. And **the reservation service is in production**.

5 故意放弃的 · Trade-offs

模型不能填字段，少见说法会被追问；「ok」不算确认

Two trade-offs. **The model can talk but can’t fill booking fields**, so unusual phrasing gets a follow-up question instead of a guess. And **“ok” doesn’t count as yes**, so some guests confirm twice.

6 适合谁 · When to use it

订错会造成真实损失的地方，就把模型挡在下单路径外

Use this design wherever **a wrong booking causes real harm**. If a mistake costs nothing, you can let the model do more.

7 我做了什么、现在到哪 · My part and status

预订服务从第一个提交写起（80 个提交里 73 个），主导 POS 后端架构，重做了 Nalu；Nalu 还在预发布测试

I wrote the reservation service from the first commit, 73 of its 80 commits, led the POS backend architecture, and rebuilt Nalu. The reservation service is live; Nalu is in pre-launch testing.

8 一句话收尾 · In one line

规则填字段，客人说「是」才下单，重试不会重复

In one line: rules fill the fields, a clear yes books it, and a retry never books twice.

关键技术深挖 · Technical deep dive

IPOT · 餐厅预订、收银与 Nalu · Restaurant platform

幂等键防重复预订 · Idempotency key on bookings

怎么做 每次下单带 **Idempotency-Key**，键存进数据库并有唯一约束：**同一个键、同样内容 → 返回第一次的结果**；同一个键、不同内容 → 409。

为什么 · 取舍 网络重试和双击都会重复发请求；靠前端防不住。**多存一张键表**换来重试安全。

追问 Q *Two identical requests arrive at the same moment. What happens?*

答 A Both try to insert the same key, and the **unique constraint** lets only one win. The loser reads the stored result and returns it, so the guest sees one booking, not two.

结账是一个事务 · Settlement in one transaction

怎么做 原来结账要调 4 个接口，中途挂掉会出现「桌子已结账但没有付款记录」。现在 **付款、小费、关单、释放桌子在同一个数据库事务里**；**菜价在下单时做快照**，改价不影响已开的单；作废有审计，超额退款等经理批。

为什么 · 取舍 钱的状态必须要么全成、要么全不成。**事务更长、锁更久**，对一家餐厅的量不是问题。

追问 Q *Why snapshot the menu price at sale time?*

答 A If the price changes while a table is still eating, the bill must not change under them. The order keeps the price it was sold at, so the receipt always matches what the guest agreed to.

Nalu：规则填字段，明确说「是」才下单 · Nalu: rules fill fields, an explicit yes books

怎么做 纯模型试点只有 17/30 全对，还会编造客人没说的值。改成 **规则读日期、时间、人数、姓名**，含糊就追问；只有 **单独、明确的「是」** 才调用下单，「ok」「好」会再问；下单后状态是待确认，等餐厅确认。

为什么 · 取舍 模型擅长聊天，不擅长精确填字段。**客人有时要多确认一次**，但不会被订错。

追问 Q *How do you know the new design is actually better?*

答 A On a separate held-out set it got **33 of 39 conversations fully right with 0 invented values**, and a live replay of 23 conversations three times ran 114 turns with 0 failures. The earlier 17 of 30 was on a different set, so I don't present it as a clean before-and-after.

Governance · AI 使用治理平台 · AI governance platform

1 谁用、什么问题 · Who and why

员工把客户数据贴进 AI 工具，安全团队看不见、拦不住、查不清

Employees paste customer data into AI tools. Security teams usually can't see it, can't stop it, and can't prove afterwards what happened.

2 它是什么 · What it is

一个平台：检测发给 AI 的个人数据，决定放行 / 打码 / 观察 / 拦截，留下改不了的记录

This is a platform that **detects personal data** in what people send to AI tools, decides whether to **allow, mask, watch or block** it, and keeps **an audit trail no one can quietly edit**.

3 怎么运作 · How it works

中英检测 → Postgres 发件箱 → 工作进程用 SKIP LOCKED 领任务 → 纯函数策略决定 → 哈希链记录

Each message goes through four steps. The detector checks it in English and Chinese. The event is written to a **Postgres transactional outbox**. A worker claims it with **SKIP LOCKED**, so two workers never take the same event, and a **pure-function policy engine** decides. The result goes into a **hash-chained history** that stores categories, hashes and positions, never the raw text.

4 三个保证 · What you can count on

双语（中文身份证召回 0 → 100%）· 同样输入同样结果 · 改一条记录就会被发现

Three things you can count on. It works in Chinese: **ID-number recall went from 0 to 100%** after I replaced patterns that never matched right after Chinese characters. The same event and policy always give the same decision. And changing any past record breaks the chain, so tampering shows.

5 故意放弃的 · Trade-offs

审计不存原文，只能看到查出了什么；用 Postgres 当队列，好运维但上限低

Two trade-offs. **The audit never stores the raw text**, so you can see what was found but not what was typed. And **Postgres is the queue**: one less system to run than Kafka, with a lower ceiling.

6 适合谁 · When to use it

小团队一个数据库就用这套；每秒百万事件再换 Kafka 这类日志

This fits a small team with one database. **At millions of events a second, I'd move to a log like Kafka.**

7 我做了什么、现在到哪 · My part and status

我一个人写的（100% 提交），参照我在 Polygraf 做过的系统重新从零实现；代码私有，检测器在 Career OPS 上线版里用

I built it alone, 100% of the commits, as a clean-room rebuild of the kind of system I worked on at Polygraf. The code is private; the detector guards the live Career OPS release.

8 一句话收尾 · In one line

先检测，决策可复现，记录谁也改不了

In one line: detect first, decide the same way every time, and keep a record nobody can quietly change.

关键技术深挖 · Technical deep dive

Governance · AI 使用治理平台 · AI governance platform

发件箱 + SKIP LOCKED 领任务 · Outbox and SKIP LOCKED

怎么做 事件先写进 **Postgres 事务性发件箱**，和业务数据同一个事务；工作进程用 **SELECT … FOR UPDATE SKIP LOCKED** 领取，互不阻塞，也不会领到同一条。

为什么 · 取舍 不用另起 Kafka，少一个系统要运维；**上限比专门的消息系统低**。

追问 Q *Why not just publish to a queue after writing to the database?*

答 A Because the write and the publish can't be atomic: the app can crash in between and lose the event. Writing the event in the **same transaction** as the data means it exists if and only if the data does.

租约过期竞争 · The lease-expiry race

怎么做 发现一个 bug：卡住的工作进程租约过期后，别的进程已经处理完同一事件，它醒来还会再写一遍。修法：**锁住这一行，只有状态还是「处理中」才继续**。

为什么 · 取舍 租约只能保证「大概率」独占；**最终正确性要靠数据库里的状态检查**。

追问 Q *How did you find it, and how do you know it's fixed?*

答 A A stalled worker could finish an event someone else had already finished. Now the worker locks the row and only proceeds while the status is still processing, so a late worker finds the event done and stops.

纯函数策略 + 哈希链审计 · Pure policy, hash-chained audit

怎么做 策略引擎是一个 **纯函数**：输入事件和策略集，输出决定，所以同样输入永远同样结果。审计记录 **每条带上一条的哈希**，只存类别、哈希和位置，不存原文；凭据用 **信封加密**。

为什么 · 取舍 不存原文更安全，**但只能看到查出了什么，看不到原话**。

追问 Q *Why does it matter that the policy engine is a pure function?*

答 A Any decision can be replayed: give it the same event and policy set and you get the same answer. That makes audits and bug reports reproducible.

中文边界问题 · Chinese boundary bug

怎么做 Presidio 用 **\b 单词边界**，紧跟在中文后面的身份证号永远匹配不上；换成不依赖单词边界的写法，再加 **校验位**。中文身份证召回 0 → 100%。

为什么 · 取舍 自己写规则要维护；**但默认规则在中文场景是错的**。

追问 Q *Why did a standard library fail on Chinese text?*

答 A \b looks for a change between word and non-word characters. Chinese characters count as word characters, so there's no boundary before the ID and the pattern never fires.

PII Detector · 浏览器内隐私检测 · In-browser PII detector

1 谁用、什么问题 · Who and why

员工把客户数据贴进 AI 工具；打码必须在文字离开设备前做，而且中英文都要管用

People paste customer data into AI tools. Masking has to happen before the text leaves their device, and it has to work for Chinese as well as English.

2 它是什么 · What it is

一个在浏览器里运行的检测器，找出卡号、邮箱、电话和 API key

This is a detector that finds card numbers, emails, phone numbers and API keys **inside the browser**, before the text goes anywhere.

3 怎么运作 · How it works

先跑规则（卡号要过 Luhn 校验）→ 可选的本地模型补人名、地址 → 同一个值用同一个占位符

It works in two passes. **Rules run first**: a card number must pass **the Luhn check**, so an order number is never called a card. Then an optional **local model running on WebGPU** adds names, addresses and dates. Repeated values keep the same placeholder, so the masked text still reads sensibly.

4 三个保证 · What you can count on

不出设备（检测时 0 个网络请求）· 中文管用（身份证召回 0 → 100%）· 快（p95 136 ms）

Three things you can count on. **0 network requests during detection**. Chinese ID-number recall went from 0 to 100%. And it's fast: **136 ms at p95** with rules plus the model, half a millisecond with rules alone.

5 故意放弃的 · Trade-offs

规则只认识见过的格式；人名地址要靠模型，首次要下载

Two trade-offs. **Rules only catch formats they know.** Names and addresses need the model, which is **a one-time download**.

6 适合谁 · When to use it

数据不能出设备就在设备上检测；可以出就用服务端，更新更方便

Use it when the data can't leave the device. **If it can, a server-side detector is easier to update.**

7 我做了什么、现在到哪 · My part and status

我一个人做的，是我治理平台里检测器的浏览器版；在线 demo 可以直接试，检测服务的代码在私有仓库

I built it alone; it's the browser version of the detector from my governance platform. The demo is live; the detector service is in a private repo.

8 一句话收尾 · In one line

先规则后模型，全程不出浏览器

In one line: rules first, the model second, and nothing leaves the browser.

关键技术深挖 · Technical deep dive

PII Detector · 浏览器内隐私检测 · In-browser PII detector

规则先跑，Luhn 校验 · Rules first, with the Luhn check

怎么做 卡号必须通过 **Luhn 校验**，所以订单号不会被当成卡号；**同一个值用同一个占位符**，遮完的文字还能读通。

为什么 · 取舍 规则在结构化数据上又快又准（只要 0.5 ms）；**没见过的格式会漏**。

追问 Q *Why not just use the model for everything?*

答 A Rules are exact and almost free on structured data. The model is for what rules can't express, like names and addresses, and it costs far more time per record.

本地模型跑在 WebGPU 上 · Local model on WebGPU

怎么做 可选的 **openai/privacy-filter 模型**，一次下载后在浏览器里用 **WebGPU** 跑；**页面检测时统计网络请求数，结果是 0**。

为什么 · 取舍 不出设备最安全；**首次下载慢，老设备可能跑不动**。

追问 Q *How can a user trust that nothing was sent?*

答 A The page counts network requests while it detects and shows the number: **0**. That's something they can check, not something they have to believe.

延迟 · Latency

怎么做 规则 + 模型 **每条 p95 136 ms**（本地 NVIDIA GB10 测），只用规则 0.5 ms。

为什么 · 取舍 多加模型换覆盖面，**慢了两百多倍**。

追问 Q *How would you keep it fast at scale?*

答 A Run the rules on everything, and send text to the model only when the rules can't decide, such as free text that may contain names.

Refund Agent · 退款智能体 · Refund agent

1 谁用、什么问题 · Who and why

让 AI 处理退款很省事，直到它为一道凉菜退了整单，或在没人批准前就告诉客人钱已退

Letting an AI agent handle refunds saves time, until it refunds a whole bill for one cold dish, or tells the customer the money is on its way before anyone approved it.

2 它是什么 · What it is

一个退款智能体：模型可以提议退款，但不能付钱

This is a refund agent where **the model can propose a refund but can’t pay one**.

3 怎么运作 · How it works

模型只能用只读工具查账 → 提议只进队列 → 固定检查 → 超过 $50 要经理批

It works in three steps. The model looks up the customer's payments with **read-only tools**; who the customer is comes from the session, so it can't reach anyone else. Its proposal **only goes into a queue**. Fixed checks then confirm the owner, whole cents, that it's still refundable and that there's a reason, and **anything over $50 needs a manager**.

4 三个保证 · What you can count on

模型动不了钱 · 不会乱承诺（没批准就说已退会被换成固定回复）· 15 个测试含全部记录回放

Three things you can count on. The model never moves money. **A reply that says the refund is done before approval is replaced with a fixed message.** And 15 tests, including a replay of every recorded run.

5 故意放弃的 · Trade-offs

退款要等人批，不能马上到账；固定回复没有模型说得自然

Two trade-offs. **Refunds wait for a person**, so customers don't get money back instantly. And **the fixed reply is less natural** than what the model would have written.

6 适合谁 · When to use it

涉及钱的事模型只提议、人来决定；不涉及钱可以放开

Use this wherever money moves: **the model suggests and a person decides**. Where no money moves, the model can do more.

7 我做了什么、现在到哪 · My part and status

我一个人做的，在线 demo，数据是合成的

I built it alone. It's a live demo on synthetic data.

8 一句话收尾 · In one line

模型提议，检查把关，人来批准

In one line: the model proposes, fixed checks filter, and a person approves.

关键技术深挖 · Technical deep dive

Refund Agent · 退款智能体 · Refund agent

读和写分开的工具 · Read tools and a propose-only tool

怎么做 **find_my_payments、get_bill 只读**；**propose_refund 只排队**，不付钱。客户身份来自会话，模型拿不到别人的账。

为什么 · 取舍 模型被提示词骗了也只能「提议」；**多一步人工**。

追问 Q *What if a prompt injection tells the model to refund someone else?*

答 A It can't: the customer comes from the session, not from anything the model says, and the only write tool just queues a proposal for a person.

固定检查 + 超额要经理 · Fixed checks and a manager threshold

怎么做 每个提议过固定检查：是本人、整数分、还能退、有理由；**超过 $50 要经理批**，异常金额会被标出。

为什么 · 取舍 检查是写死的，**不灵活但可预测**。

追问 Q *Why a fixed $50 threshold?*

答 A It's a policy number the business can change, but it has to be a rule in code, not something the model decides case by case.

不让模型乱承诺 · No false promises

怎么做 **批准前回复里说「已退款」，就整句替换成固定模板**；**回放全部录下的运行**做测试，15 个测试。

为什么 · 取舍 **固定模板不自然**，但不会骗客人。

追问 Q *How do you test an LLM agent deterministically?*

答 A I record real model runs and replay them in tests, so the checks run against what the model actually said, every time, without calling the model.

Dispute Ingest · 信用卡争议接入 · Card dispute ingestion

1 谁用、什么问题 · Who and why

发卡机构收到的争议会被卡组织重复推送，而且每件都有监管截止日期

Card issuers receive disputes as network files and webhooks, often more than once, and each one starts a regulatory deadline. Crediting a fraudulent claim by mistake is money that doesn't come back.

2 它是什么 · What it is

一个 Go 服务：每个案子一条记录、一个明确的截止日期，有风险的交给人

This is a Go service that turns all of that into **one record per case**, **one dated deadline**, and **a person in the loop for anything risky**.

3 怎么运作 · How it works

统一原因码、金额转成整数分 → 按卡组织 + 案号去重 → 算 Reg E / Reg Z 截止日 → 只有低风险的自动入账

Each dispute goes through four steps. Visa and Mastercard reason codes are mapped to one model, and amounts become **integer cents**. Ingestion is **idempotent on network plus case ID**: a repeat returns 200, the same ID with different content returns 409. It computes **Reg E and Reg Z deadlines**. Only low-risk, in-window cases under a limit are credited automatically.

4 三个保证 · What you can count on

不重复（20 次并发重复推送 → 1 条记录）· 截止日可见 · 风险交给人

Three things you can count on. **20 concurrent redeliveries of one case create exactly 1 record.** Every case has a visible deadline. And fraud signals always go to a person.

5 故意放弃的 · Trade-offs

没有自动拒绝，人要看更多案子；截止日规则是简化版

Two trade-offs. **There is no auto-deny**, so people review more cases. And **the Reg E and Reg Z rules are simplified**.

6 适合谁 · When to use it

自动化只做低风险那一头；拒绝客户永远交给人

Automate only the safe side. **Saying no to a customer should stay with a person.**

7 我做了什么、现在到哪 · My part and status

我一个人写的开源原型，不是公司项目；本地运行，没有部署，数据是合成的

I wrote it alone as an open-source prototype, not employer work. It runs locally on synthetic data and isn't deployed.

8 一句话收尾 · In one line

一案一条记录、一个期限，有风险的交给人

In one line: one record per case, one deadline, and a person for anything risky.

关键技术深挖 · Technical deep dive

Dispute Ingest · 信用卡争议接入 · Card dispute ingestion

按卡组织 + 案号幂等 · Idempotent on network plus case ID

怎么做 同一个案子重复推送返回 **200**，同号不同内容返回 **409**；**20 次并发重复推送 → 1 条记录**。

为什么 · 取舍 重复是常态；**409 需要人看**。

追问 Q *Why 409 instead of just overwriting with the newer copy?*

答 A Different content under the same case ID means something is wrong upstream. Overwriting would hide it; a 409 makes someone look.

并行校验、串行提交 · Parallel validation, serialized commits

怎么做 金额解析成 **整数分**，每行的字段错误进隔离日志；校验并行跑，**提交串行**，写进 **哈希链账本**，启动时校验，被改过就拒绝启动。

为什么 · 取舍 串行提交慢一点，**换来账本顺序确定**。所有测试都在 go test -race 下通过。

追问 Q *Why refuse to start if the ledger was edited?*

答 A If the history can't be trusted, any decision based on it can't either. Stopping loudly is better than running on bad data.

只自动处理低风险 · Automate only the safe side

怎么做 简化版 **Reg E（借记卡）和 Reg Z（信用卡）** 截止日；超出 120 天窗口的会被标出；**只有低风险、窗口内、金额低于上限的自动入账**；**没有自动拒绝**。

为什么 · 取舍 自动入账错了还能追回；**自动拒绝错了伤客户**。

追问 Q *Why no auto-deny?*

答 A A wrong denial harms a real customer and may break the regulation. Those decisions stay with a person.

Loan Draw Review · 放款审核控制 · Loan-draw review controls

1 谁用、什么问题 · Who and why

建筑贷款按阶段放款；AI 能发现缺发票，但打钱的决定必须留给人，也绝不能重复放款

Construction loans are paid out in stages called draws. An AI check can flag a missing invoice, but the decision to wire money has to stay with people, and a retry must never fund the same draw twice.

2 它是什么 · What it is

借贷产品里的一组控制：两人签字才放款、同一笔不会放两次、每步可验证

In a team lending product, I built the controls around payouts: **money leaves only after two people sign off**, **the same draw can’t be funded twice**, and every step is recorded in a verifiable audit chain.

3 怎么运作 · How it works

高风险的要两人审批 → 放款时锁住这笔贷款，电汇号唯一 → 哈希链记录 → 崩溃后用 Temporal 接着跑

For a higher-risk draw, **two different people must approve**, behind a feature flag. Funding takes **a row lock on the loan**, checks for a repeat, and relies on **a unique index on the wire reference**. Each step is added to a **hash-chained audit record**, and verification runs through **Temporal**, so a crash resumes instead of repeating.

4 三个保证 · What you can count on

不会重复放款（103 个真库金融测试，含并发审批）· 有人签字 · 发布安全（4/4 次故障演练）

Three things you can count on. No double funding, shown by **103 financial tests on a real PostgreSQL database, including concurrent approvals**. Two people sign. And a release gate that passed **4 of 4 failure rehearsals**.

5 故意放弃的 · Trade-offs

多一个签字人会让放款变慢

The main trade-off: **a second signer slows payouts down**.

6 适合谁 · When to use it

钱出去就收不回，慢一点值得

For money, that's the right trade, because **a payout can’t be pulled back once it’s sent**.

7 我做了什么、现在到哪 · My part and status

我写了审核控制、流程可靠性和发布安全（5 个 PR）；AI 标记部分是同事做的；已合并，还没上线

I wrote the review controls, workflow reliability and release safety in 5 pull requests; the AI flagging is my teammates' work. It's merged and not yet deployed to production.

8 一句话收尾 · In one line

两人签字，一笔一次，每步留痕

In one line: two signatures, one payout per draw, and a record of every step.

关键技术深挖 · Technical deep dive

Loan Draw Review · 放款审核控制 · Loan-draw review controls

双人审批 · Two-person approval

怎么做 高风险放款要 **制单人和复核人是两个不同的人**（maker-checker），放在功能开关后面。

为什么 · 取舍 **多一个签字人更慢**，但钱出去就收不回。

追问 Q *How do you stop the same person approving twice under two roles?*

答 A The approval records who made and who checked the request, and the check rejects a checker who is the same person as the maker.

不会重复放款的三层保护 · Three layers against double funding

怎么做 **贷款级行锁** → **重复放款检查** → **电汇号上的部分唯一索引**。103 个金融测试跑在真 PostgreSQL 上，包括并发审批。

为什么 · 取舍 应用层检查在并发下会漏，**最后一道必须在数据库里**。

追问 Q *Why a partial unique index rather than a plain one?*

答 A Only funded draws carry a wire reference, so the index applies to those rows only; it guarantees one wire per draw without blocking draws that aren't funded yet.

发件箱 + Temporal 恢复；发布闸门 · Outbox, Temporal recovery and a release gate

怎么做 核验通过 **事务性发件箱** 和 **Temporal** 跑，崩溃后接着跑；部分唯一索引防两次核验同时跑。替换掉会掩盖迁移失败的旧部署流程：**先验证备份 → 预检 → 迁移 → 健康检查 → 回滚**，再加一个有新失败测试就拦住的门槛；4/4 次故障演练通过。

为什么 · 取舍 发布更慢，**但失败的迁移不会再悄悄上线**。

追问 Q *What did the four release rehearsals test?*

答 A A failed migration, an API that couldn't boot, a failing preflight check, and a database not set to UTC. All four were caught and rolled back.

Career OPS · 求职数据与面试练习 · Job market feed

1 谁用、什么问题 · Who and why

找工作时用的第三方职位源标成「最新」的，只有 16% 真是最近发的

When I was job hunting, a third-party job feed labelled its rows “recent.” Only 16% of them, 214 of 1,375, were actually posted in the last three days.

2 它是什么 · What it is

一个可以追溯到源头的职位数据流，加上按简历出题的面试练习

So I built a job feed taken **straight from employers’ own hiring systems**, where every row is traceable, plus interview practice based on my résumé. It extends the open-source career-ops project.

3 怎么运作 · How it works

从招聘系统接口抓 → 记下日期来自哪个字段 → 连续两次缺失才算关闭 → 公开前过隐私检查

It works in four steps. Postings come from **ATS APIs like Greenhouse, Lever and Ashby**, keeping the source URL and **which field each date came from**. Nothing is deleted: a posting closes only after it's **missing twice in a row**. Before anything is public, it passes **the bilingual PII detector and a fail-closed scan**.

4 三个保证 · What you can count on

可追溯（一次快照 264,148 条）· 日期可信 · 隐私检查上线前拦下 5 个问题

Three things you can count on. Every row is traceable: **264,148 postings** in one snapshot, each with its source. The dates are honest. And the privacy gate **caught 5 issues before launch**.

5 故意放弃的 · Trade-offs

只覆盖有公开招聘接口的公司；判断关闭要多等一轮

Two trade-offs. **It only covers companies with public ATS APIs.** And **closing a posting waits an extra round**, so a filled job can look open a little longer.

6 适合谁 · When to use it

求职时宁可少而准，不要多而乱

For a job search, **fewer accurate rows beat a bigger, noisier feed**.

7 我做了什么、现在到哪 · My part and status

我在开源项目上做了数据流、简历工具和公开发布（我写的部分 439 个测试）；只读看板在我的 Azure 服务器上

On top of the open-source project, I built the market-feed pipeline, the résumé tooling and the public release, with 439 tests in the parts I wrote. A read-only dashboard runs on my Azure server.

8 一句话收尾 · In one line

数据来自源头，每条可追溯，公开前先把关

In one line: data from the source, a trail for every row, and a check before anything goes public.

Think in trade-offs ☺

关键技术深挖 · Technical deep dive

直接从招聘系统抓，记下日期来源 · Collect from ATS APIs, keep the date's source

怎么做 从 **Greenhouse、Lever、Ashby 等 ATS 接口** 抓，保留源链接和 **每个时间戳来自哪个字段**。

为什么 · 取舍 第三方源说「最新」的只有 16% 是真的；**只覆盖有公开接口的公司**。

追问 Q *How did you find out the third-party feed was wrong?*

答 A I compared its “recent” rows with the employers' own posting dates: only 214 of 1,375 had been posted in the last three days.

从不删除，连续两次缺失才关 · Never delete; close after two misses

怎么做 职位只有 **连续两次抓不到** 才标记关闭，所有状态变化都留着。

为什么 · 取舍 一次抓取失败不会误关；**关闭会晚一轮**。

追问 Q *Why not close a posting the first time it disappears?*

答 A A single missed fetch is often a network or API hiccup. Two misses in a row is much stronger evidence the job is really gone.

发布前失败即关的隐私闸门 · A fail-closed privacy gate before release

怎么做 公开字段要过已知字段规则、**双语 PII 检测器** 和一个 **失败即关的扫描**；上线前拦下 5 个问题。

为什么 · 取舍 **有疑问就不发布**，会多拦一些正常数据。

追问 Q *What does fail-closed mean here?*

答 A If the scan errors or can't decide, the field is not published. Missing a field is better than leaking one.

Remote Commander Ops · 远程控制守护 · Remote access supervision

1 谁用、什么问题 · Who and why

我要从手机上的 AI 聊天随时连回家里的 Mac；进程在跑不代表连接还活着

I need to reach my home Mac from an AI chat app on my phone at any time. The remote-access process can be running while the connection behind it is dead.

2 它是什么 · What it is

一套守护和监控：保证只跑一个实例、安装安全、断了能说出为什么

This is a supervision and monitoring setup that keeps **exactly one remote-access process running**, installs safely, and **says why access is down**.

3 怎么运作 · How it works

launchd 托管一个实例 → 安装器先只读检查、失败自动恢复 → 每分钟根据元数据判断属于 6 类中的哪一类

It has three parts. **launchd** runs one supervised instance and refuses a second one, because two instances rotate the same sign-in token and break each other. The installer has a **read-only check mode** and restores itself if the new job is rejected. Every minute, a monitor sorts health into **6 classes using metadata only**: ok, process, auth, relay, network or unknown.

4 三个保证 · What you can count on

一直在线 · 告诉你原因 · 160 个测试；独立审查提的 10 条改了 9 条

Three things you can count on. It stays up. It tells you the reason, not just “down.” And it's tested: **160 tests** against stubbed system tools, with 9 of 10 independent review findings fixed before merge.

5 故意放弃的 · Trade-offs

只看元数据，有些故障只能判为「未知」；连续 3 次失败才告警，会晚几分钟

Two trade-offs. It reads **metadata only, never commands or tokens**, so some failures stay “unknown.” And it alerts **after three failures in a row**, so you hear a few minutes later.

6 适合谁 · When to use it

个人机器上，隐私和少误报比早一分钟告警更重要

On a personal machine, **privacy and fewer false alarms matter more than an alert one minute sooner**.

7 我做了什么、现在到哪 · My part and status

我一个人做的，包了一层开源的 Desktop Commander，没改它的代码；10-10 部署在我的 Mac 上

I built it alone as a wrapper around the open-source Desktop Commander, without forking it. It has run on my Mac since October 10.

8 一句话收尾 · In one line

只跑一个，装得安全，断了说清楚原因

In one line: one supervised process, a safe install, and a clear reason when it breaks.

关键技术深挖 · Technical deep dive

Remote Commander Ops · 远程控制守护 · Remote access supervision

只允许一个实例 · Exactly one supervised instance

怎么做 厂商进程持有轮换的 OAuth 会话；**起两个会轮换同一个 refresh token，互相踢掉**。launchd 托管一个，终端里已有一个就等。**按完整命令行匹配进程**，修掉了带空格路径和全路径 node 漏掉的 bug。

为什么 · 取舍 **手动在终端起的会被让路**。

追问 Q *Why match on the full command line?*

答 A Matching on the process name missed real cases, such as a full-path node binary or a path with spaces, and that let a second instance start.

只看元数据的健康分类 · Health classes from metadata only

怎么做 每分钟分成 6 类：ok / process / auth / relay / network / unknown。**数被托管进程到中继端口的已建立连接**；连接没了才探网络；**看会话文件的年龄，对照厂商 45 分钟刷新周期**判断登录是否过期。

为什么 · 取舍 **不读命令和 token**，所以有的故障只能判 unknown。

追问 Q *How do you tell an auth failure from a network failure without reading the token?*

答 A If the session file is older than the vendor's 45-minute refresh, it's auth. If the session is fresh but there's no connection to the relay and the network probe fails, it's network.

awk 和 Python 两份匹配器对账 · A parity test between awk and Python

怎么做 启动脚本用 awk、监控用 Python 匹配进程；**同一组样例两边都跑**，抓到一处两边结论不同。安装器有 **--check 只读模式**，失败自动恢复；160 个测试用假的 launchctl、ps、plutil。

为什么 · 取舍 两份实现要维护，**所以必须有对账测试**。

追问 Q *Why test against stubbed system tools?*

答 A So every install path, including failures and rollbacks, can be tested on any machine without touching the real launchd.

双语论文朗读器 · Bilingual Note Reader

1 谁用、什么问题 · Who and why

我的笔记和论文中英混排；单一声音要么念坏中文要么念坏英文，云端语音按字收费还要把私密笔记传出去

My notes and papers mix Chinese and English. A single voice mangles one language or the other, and cloud speech APIs charge per character and send private notes off my machine.

2 它是什么 · What it is

一个在 Obsidian 里朗读笔记和 PDF 的工具，中英文各用母语声音，正在读的句子会高亮

So I built a reader for my notes and PDFs in Obsidian that **switches to a native voice for each language** mid-sentence and highlights the sentence being read, **all on my own hardware**.

3 怎么运作 · How it works

按文字切分 → 中文送 GPU 上的 Qwen3-TTS、英文送 CPU 上的 Kokoro → 音量对齐、停顿定长 → 失败一次就静音跳过

Each line is **split by script**: Chinese goes to **Qwen3-TTS on the GPU**, English to **Kokoro on the CPU**. A Chinese sentence is sent whole so the intonation stays natural. Clips are matched in loudness, pauses are set to exact lengths, and a failed piece is retried once and then replaced by silence, so one bad sentence never stops a long note.

4 三个保证 · What you can count on

两种语言都自然 · 不用等（每行都快过实时）· 0 次付费调用

Three things you can count on. Natural voices in both languages. No waiting: **every line is generated faster than real time**, and a cached line replays in 0.07 s. And **0 paid API calls**.

5 故意放弃的 · Trade-offs

需要一台带 GPU 的机器；中文模型挂了会退回音质差一点的声音

Two trade-offs. It needs **a GPU machine at home**. And if the Chinese model is down, **Chinese falls back to a lower-quality voice** instead of stopping.

6 适合谁 · When to use it

私密笔记天天读就值得自己跑；偶尔用用云端语音就行

If you read private notes every day, **running it yourself pays off**. For occasional use, a cloud voice is fine.

7 我做了什么、现在到哪 · My part and status

我一个人做的，改了开源的 Narrate 插件；在家里的服务器上天天在用，只在私有网络里

I built it alone, patching the open-source Narrate plugin. I use it every day on a home server that's only reachable on my private network.

8 一句话收尾 · In one line

先按语言切开，再给每部分最好的声音

In one line: split by language first, then give each part its best voice.

Think in trade-offs ☺

关键技术深挖 · Technical deep dive

双语论文朗读器 · Bilingual Note Reader

先按文字切分，再合成 · Split by script before synthesis

怎么做 语音模型按语言训练，**混合文本要在合成前切开**：汉字一段给中文模型，拉丁字母给英文模型；**中文句子整句送，不在逗号处切**，语调连贯。

为什么 · 取舍 切分规则简单可预测；**少数混写词会切得不完美**。

追问 Q *Why not use one multilingual voice?*

答 A A single voice mangles one language or the other. Splitting lets each part use a native voice, and keeping Chinese sentences whole keeps the intonation natural.

确定的停顿和不卡顿 · Deterministic pauses and gapless playback

怎么做 每段先裁掉首尾静音，再插 **精确的间隔**（句末 10 ms，英文逗号 500 ms）；插件 **边播边合成后面 3 段**，每行都快过实时。失败重试一次就换成静音。

为什么 · 取舍 **一句失败会变成静音**，但长文不会中断。

追问 Q *How is there no waiting between lines?*

答 A While one chunk plays, the next three are already being synthesized, and every line synthesizes faster than real time, so after the first line the audio is always ready.

PDF 上精准高亮 · Exact highlights on the PDF

怎么做 直接读 **PDF 查看器的文本层**：每个句子记住它来自哪段字符，所以高亮能落在正在读的字上，跨行跨页都准；跳过页眉、页码、引用、图表和参考文献。

为什么 · 取舍 依赖 PDF 有文本层，**扫描件不行**。

追问 Q *How do you keep the highlight on the right words?*

答 A Each sentence keeps the exact character span it came from in the viewer's text layer, so the highlight is drawn on those characters, not guessed from the audio.

REEA · AI 房产平台 · AI real-estate platform

1. **谁用、什么问题** · Who and why
   买家说不清想要什么，经纪人顾不过来，房源和资料东一处西一处
   Home buyers struggle to pin down what they want, agents have little time for each one, and listings and paperwork are scattered.

2. **它是什么** · What it is
   一个给买家、卖家和经纪人的房产平台，加一个用 GPT-3 写房源内容的工具
   REEA was a real-estate platform for buyers, sellers and agents, with **a GPT-3 tool that writes listing content**.

3. **怎么运作** · How it works
   Django 账户、结构化房源和搜索 → 写房源时三步 GPT-3：选题 → 小标题 → 正文 → 按订阅限每月字数
   Agents sign in and post structured listings (type, rooms, amenities, price, video), and buyers search them. To write a listing, **three GPT-3 steps** go from topic ideas to section titles to finished sections, with **a monthly word allowance per subscriber**.

4. **三个保证** · What you can count on
   房源和搜索在一个地方 · 写房源更快 · 我写了 95 个提交里的 68 个
   Three things it delivered: listings and search in one place, faster listing copy, and a product I built hands-on, **68 of the 95 commits**.

5. **故意放弃的** · Trade-offs
   早期产品，现在没在运行；当时没写测试
   Two honest limits. It was **an early-stage product that isn’t running today**. And **we shipped without tests**, which is the first thing I'd change.

6. **适合谁** · When to use it
   今天重做会先写测试再加 AI
   If I rebuilt it today, I'd **write the tests before the AI features**.

7. **我做了什么、现在到哪** · My part and status
   我是创始工程师：搭平台、带合同开发者、参与产品规划和融资路演材料
   I was the founding engineer: I built the platform, worked with contract developers, and helped with the product plan and pitch deck.

8. **一句话收尾** · In one line
   房源、搜索和 AI 写稿在一个地方，教训是先写测试
   In one line: listings, search and AI copy in one place, and the lesson to test first.

关键技术深挖 · Technical deep dive

REEA · AI 房产平台 · AI real-estate platform

三步 GPT-3 写房源 · Three GPT-3 steps for listing copy

怎么做 **选题 → 小标题 → 正文**，每步一个调用，每个订阅用户每月有字数上限。

为什么 · 取舍 拆步骤更可控；**调用次数多**。

追问 Q *Why split it into three calls instead of one?*

答 A Each step is smaller and easier to check, and the user can pick or edit between steps instead of getting one long block.

没写测试的教训 · The cost of shipping without tests

怎么做 和合同开发者一起做，95 个提交里我写了 68 个；**当时没写测试**，改功能就容易坏别的地方。

为什么 · 取舍 现在先写测试，再加 AI 功能。

追问 Q *What would you do differently?*

答 A Write tests for the core flows first, accounts, listings and search, so the AI features could change quickly without breaking them.

LightUp · 看得见影响的捐赠 · Giving that shows impact

1 谁用、什么问题 · Who and why

年轻捐款人想看到真实影响，却很少知道钱去了哪；慈善机构也很难留住他们

Young donors want to see real impact, but rarely know where their money goes, and charities struggle to keep them engaged.

2 它是什么 · What it is

一个捐赠应用：把捐款人和项目配对，告诉每个人他改变了谁的生活，也把慈善机构连起来

LightUp is a giving app that **matches donors to causes**, tells each donor **whose life their gift changed**, and connects the charities behind it.

3 怎么运作 · How it works

按兴趣和价值观配对 → 捐款 → 在项目地图上收到受助人的信、语音和照片

A donor picks interests and values, and the app **matches them to causes**. After giving, they see funded projects on a map and get **letters with voice messages and photos** from the people helped.

4 三个保证 · What you can count on

看得见影响 · 故事是为你写的，而且来自机构的真实报告 · 机构之间能互相找到

Three things it offers. Impact you can see. A story written for you, **based on what the charity actually reports**. And a shared directory where charities can find each other.

5 故意放弃的 · Trade-offs

2023 年 AI 还写不好个性化的信，所以停过；2026 年重启，还在开发

Two honest limits. In 2023, **AI couldn’t yet write those letters well**, so the project paused. It restarted in 2026 and is **still in development**.

6 适合谁 · When to use it

影响故事必须来自机构真实报告，不能让模型编

An impact letter only works if it's true, so **the model writes from the charity’s report, never from imagination**.

7 我做了什么、现在到哪 · My part and status

我是联合创始人；2023 年做了概念和 React 原型，2024 年做了高保真原型和概念视频，拿了伦敦设计奖金奖

I'm a co-founder. In 2023 we defined the concept and started a React prototype; in 2024 we built a high-fidelity mobile prototype and concept video, which won Gold at the London Design Award.

8 一句话收尾 · In one line

配对合适的项目，讲真实的影响，让机构连起来

In one line: the right cause, a true story of impact, and charities that work together.

关键技术深挖 · Technical deep dive

LightUp · 看得见影响的捐赠 · Giving that shows impact

影响故事只来自机构的真实报告 · Impact letters grounded in charity reports

怎么做	每封信 **只用慈善机构实际报告的内容** 生成，不让模型想象。
为什么 · 取舍	**报告少的项目信也会短**，但都是真的。
追问 Q	*How do you stop the model from inventing an impact story?*
答 A	It only writes from the charity's report, and anything that isn't in the report doesn't go in the letter.

按兴趣和价值观配对 · Matching on stated interests

怎么做	捐款人自己选兴趣和价值观，**按这些配对项目**；机构之间有共享目录。
为什么 · 取舍	只用用户自己说的偏好，**推荐可能不够惊喜**，但不需要追踪用户。
追问 Q	*Why only stated preferences?*
答 A	It's transparent and needs no tracking; donors know why they see a cause.

Polygraf.ai 实习 · PII 脱敏与治理后台 · Polygraf internship

1 **谁用、什么问题** · Who and why

企业员工把数据贴进大模型；进模型之前要先找出并遮住个人信息

Polygraf sells data-privacy tools for companies using large language models. Before text reaches an LLM, personal information in it has to be found and masked.

2 **它是什么** · What it is

实习期间做了两样：LLM 输入的 PII 检测脱敏原型，和一个治理后台

As an AI engineer intern I worked on two things: **a PII detection and masking prototype for LLM inputs**, and **a governance dashboard** for privacy policies, credentials and security incidents.

3 **怎么运作** · How it works

Presidio 规则 + Hugging Face NER 模型 + 规则校验 → 遮住再交给模型；后台用 React / TypeScript + FastAPI

The prototype combines three layers: **Presidio** recognizers, a **Hugging Face Transformers NER model**, and rule-based checks, then masks what they find before the text goes to the model. The dashboard is **React and TypeScript on a FastAPI backend**.

4 **三个保证** · What you can count on

规则抓结构化的、模型抓人名地址 · 管理员能在一个后台管策略、凭据和事件

What it gave the team: structured data like IDs caught by rules, names and addresses caught by the model, and one place for admins to manage policies, credentials and incidents.

5 **故意放弃的** · Trade-offs

后来发现 Presidio 的边界写法对紧跟中文的号码不起作用

The lesson I took away: **Presidio’s default patterns never match an ID written right after Chinese text**, which I only found and fixed later in my own rebuild.

6 **适合谁** · When to use it

要做中英双语就不能直接用默认规则

So for bilingual products, **default detectors need their own tests in each language** before you trust them.

7 **我做了什么、现在到哪** · My part and status

我主动争取做了 PII 原型（BQ 故事 S15）；后来在自己的治理平台里从零重写了这类系统

I asked to build the PII prototype, which is my BQ story S15. Later I rebuilt this kind of system from scratch in my own governance platform.

8 **一句话收尾** · In one line

先找出个人信息、遮住，再交给模型

In one line: find personal data, mask it, and only then hand the text to a model.

关键技术深挖 · Technical deep dive

Polygraf.ai 实习 · PII 脱敏与治理后台 · Polygraf internship

三层检测 · Three detection layers

怎么做 **Presidio** 识别器 + **Hugging Face Transformers NER** + 规则校验，找到后遮住再交给模型。

为什么 · 取舍 多层覆盖更全，**也更慢、更难调**。

追问 Q *Why combine three approaches?*

答 A Rules are exact on structured data, the NER model finds names and places, and Presidio ties them together with a common format for findings.

治理后台 · Governance dashboard

怎么做 **React / TypeScript + FastAPI**，管理隐私策略、凭据和安全事件。

为什么 · 取舍 我参与其中一部分；**不是我一个人做的**。

追问 Q *What was your part?*

答 A I built the PII prototype, which I asked to own, and worked on parts of the dashboard.

开放投融资数据 · Open Private Markets

1 谁用、什么问题 · Who and why

PitchBook 这类私募数据很贵；但美国 SEC 公开数据里已经有大量新基金和投顾信息

Private-markets data like PitchBook is expensive. Yet the SEC already publishes a lot of it for free: every new private fund files a Form D, and advisers file Form ADV.

2 它是什么 · What it is

只用公开数据，重建分析师在 PitchBook 里做的事，做成只读、可重置的环境

This project rebuilds what an analyst does in a product like PitchBook **using only open data**: SEC Form D and Form ADV, public domain or CC0. It's built as **a read-only environment that resets from a fixed snapshot**.

3 怎么运作 · How it works

SEC 数据 → 固定快照（DuckLake）→ 只读 API 和 MCP 工具 → 任务集 + 确定性校验器

It works in layers. SEC filings are loaded into **a fixed DuckLake snapshot**. On top sit **read-only tools, as an API and an MCP server**. Then come analyst tasks, each checked by **a deterministic verifier**, plus a reference solver and a cost log.

4 三个保证 · What you can count on

数据合法可公开 · 每次结果可复现 · 第一份试点清单：2026Q3 新成立私募基金 5,780 只

Three things you can count on. The data is legal to share. Every run is reproducible from the same snapshot. And it already produces something useful: **a list of 5,780 private funds launched in Q3 2026**, by type, with links to the SEC filings.

5 故意放弃的 · Trade-offs

不抓 PitchBook、Crunchbase、LinkedIn；现在只有季度数据，周报会晚四周以上

Two trade-offs. **It never scrapes PitchBook, Crunchbase or LinkedIn**, so some fields are missing. And **the data is quarterly for now**, so a weekly digest runs more than four weeks behind.

6 适合谁 · When to use it

要合规、可复现就用公开数据；要最快最全还是得买商业数据

It suits people who need data they can legally reuse and reproduce. **If you need the fastest, most complete coverage, a commercial database still wins.**

7 我做了什么、现在到哪 · My part and status

我一个人在做；本地原型，第一份试点清单已备好，还没和客户聊

I'm building it alone. It's a local prototype; the first pilot list is ready and the first customer conversation hasn't happened yet.

8 一句话收尾 · In one line

用公开数据重建私募数据产品，可复现、可合法分享

In one line: rebuild a private-markets data product from open data, so it's reproducible and legal to share.

关键技术深挖 · Technical deep dive

开放投融资数据 · Open Private Markets

固定快照 + 只读工具 · A fixed snapshot and read-only tools

怎么做 SEC 数据装进 **DuckLake 固定快照**，每次从快照重置；上面只开 **只读的 API 和 MCP 工具**。

为什么 · 取舍 可复现、可审计；**数据不是实时的**。

追问 Q *Why reset from a fixed snapshot?*

答 A So every task and every answer can be reproduced exactly; the verifier checks against the same data every time.

确定性校验器 · Deterministic verifiers

怎么做 每个分析任务都有 **确定性校验器**，给出对或错，不靠模型打分；还有参考解题器和单环境成本记录。

为什么 · 取舍 写校验器费时，**但结果可信**。

追问 Q *Why not use a model to grade answers?*

答 A A model grader can be wrong or inconsistent. A deterministic check gives the same verdict every time, which is what makes the environment usable for evaluation.

新基金名单怎么定义 · How a new fund is defined

怎么做 **Form D 新申报，且首笔出售在 12 个月内或还没发生**；排除货币市场和流动性基金、单笔超过 $50B 的异常值；**不含人名和地址**。2026Q3 共 5,780 只。

为什么 · 取舍 **只有季度数据**，周报会晚四周以上。

追问 Q *How do you make sure the list doesn't include old funds?*

答 A A fund counts as new only if it's a new filing and its first sale is within the last 12 months or hasn't happened yet.

AI Infrastructure Commander · 自建 MCP 工具服务器（规划中）· Self-hosted MCP tool server

1. **谁用、什么问题** · Who and why
   我想在 ChatGPT 或 Claude 里直接让 AI 查看、维护我的 Mac 和 GPU 主机，但不能让 AI 自己决定危险操作
   I want to ask ChatGPT or Claude to check and maintain my Mac and GPU machine from the chat apps I already use, without letting the AI decide on risky actions by itself.

2. **它是什么** · What it is
   一个自建的 MCP 工具服务器；不做聊天界面，不做自己的模型路由
   Kai Commander is **a self-hosted MCP tool server**. It doesn't add a new chat app or model router; it gives the existing ones a limited set of tools.

3. **怎么运作** · How it works
   ChatGPT / Claude → MCP 连接 → Commander（权限、审批、日志）→ Mac 本地执行器或 SSH 到 GPU 主机
   A request flows like this: ChatGPT or Claude calls **the Commander MCP server**, which checks **device policy, approval and logging**, then hands the action to a local executor on the Mac or over **SSH** to the GPU machine. Restarts, deletes, deploys and config changes always need explicit approval.

4. **三个保证** · What you can count on
   两边聊天都能用 · 危险操作必须人批 · 每步有日志
   Three things it's designed to guarantee. It works from both ChatGPT and Claude. Anything risky waits for a person. And every action is logged.

5. **故意放弃的** · Trade-offs
   两家平台的连接方式不同，要分别部署；Claude 网页版需要公网 HTTPS 入口
   Two trade-offs. **Each platform connects differently**, so there are two entry points to deploy and secure. And **Claude's web app needs a public, authenticated HTTPS endpoint**, so a private-network address alone isn't enough.

6. **适合谁** · When to use it
   只读诊断先上，写操作等审批流程稳定后再开
   The plan is to **ship read-only checks first** and open write actions only once approvals are proven.

7. **我做了什么、现在到哪** · My part and status
   产品范围 v0.4 已确定，还没部署任何服务；它和已上线的 Remote Commander Ops 是两件事
   The product scope, version 0.4, is approved; no service is deployed yet. It's separate from Remote Commander Ops, which is already running.

8. **一句话收尾** · In one line
   在已有的聊天里加一套受限的工具，危险的事永远先问人
   In one line: add limited tools to the chat apps I already use, and always ask a person before anything risky.

关键技术深挖 · Technical deep dive

AI Infrastructure Commander · 自建 MCP 工具服务器（规划中）· Self-hosted MCP tool server

MCP、执行器、Skill 的分工 · MCP server, executor and skill

怎么做 **MCP 服务器** 向 AI 平台暴露受限工具；**本地执行器或 SSH** 真正执行；Skill 只教 AI 怎么用工具，可选。**明确不做聊天界面和模型路由**。

为什么 · 取舍 范围小，才做得完。

追问 Q *Isn't a skill enough?*

答 A No. A skill only teaches the AI how to use tools; it doesn't connect to anything. The MCP server is what the chat app actually calls.

两家平台连接方式不同 · Two platforms, two connection paths

怎么做 ChatGPT 自定义 MCP 只支持网页、走 **Secure MCP Tunnel**；Claude 网页和手机需要 **公网可达、有认证的 HTTPS**；Claude Desktop 可以走本地 stdio。

为什么 · 取舍 **要分别部署和保护两个入口**。

追问 Q *Why can't Claude just use your Tailscale address?*

答 A Claude's custom connectors are called from Anthropic's cloud, so the endpoint must be reachable from the internet with authentication; a private Tailscale address isn't.

审批不靠 AI 判断 · Approvals don't depend on the AI

怎么做 重启、删除、部署、改生产配置，**工具自己校验权限并要求明确批准**，不因为 AI 觉得可以就执行。

为什么 · 取舍 **多一步确认**。

追问 Q *Why check permissions in the tool rather than in the prompt?*

答 A A prompt can be talked around. A check in the tool can't, whatever the model was told.

硕士论文重做 · Revisiting my MSc thesis

1 谁用、什么问题 · Who and why

我的硕士论文说混合模型预测股价最好；今年重跑发现它连「明天 = 今天」都赢不了

My master's thesis said a hybrid model forecast stock prices best. When I re-ran it this year, it couldn't beat the simplest guess: tomorrow's price equals today's.

2 它是什么 · What it is

一次诚实的复盘：重现原结果，找出问题，重新设计实验

This is an honest re-run: **reproduce the original exactly**, measure every problem, then redesign the study.

3 怎么运作 · How it works

原代码三个种子重现 → 列出 12 个问题逐个量化 → 20 只股票、208 个起点重做 → 16 个模型都和随机游走比

First I reproduced the original code with three seeds, matching the thesis's own result files. Then I listed and measured **12 problems**, including **no random-walk baseline**, **a scaler fitted on test data** and a 60-day misalignment. Finally I rebuilt the study: 20 tickers, 208 forecast origins, 1, 5 and 20 days ahead, and **16 models tested against the random walk with Diebold-Mariano tests**.

4 三个保证 · What you can count on

原混合模型误差是「明天 = 今天」的 7–13 倍 · 16 个模型 0 个显著更好 · 波动率倒是能预测（误差 −20%）

What I found: the thesis's hybrid had **7 to 13 times the error** of “tomorrow equals today.” **0 of 16 models** beat the random walk significantly. But volatility was predictable, with **20% lower error** than the naive baseline.

5 故意放弃的 · Trade-offs

承认原结论站不住；重做花了大量时间但没有「更好的模型」可以展示

The trade-off: **I had to say my own conclusion didn’t hold**, and all that work produced no “better model” to show off.

6 适合谁 · When to use it

预测类问题先和最简单的基线比，再谈模型

My take: **always compare against the simplest baseline first**, before trusting any model.

7 我做了什么、现在到哪 · My part and status

论文和重做都是我一个人；实验已完成，代码在本地没发布

I'm the sole author of both the thesis and the re-run. The experiments are complete; the code is local and not yet published.

8 一句话收尾 · In one line

先和最简单的猜测比，再相信模型

In one line: beat the simplest guess first, then trust the model.

关键技术深挖 · Technical deep dive

硕士论文重做 · Revisiting my MSc thesis

先和随机游走比 · Compare against the random walk first

怎么做 原论文没有「明天 = 今天」这个最简单的基线；加上之后，原混合模型误差是它的 **7–13 倍**。

为什么 · 取舍 **最难接受的是自己的结论站不住**。

追问 Q *Why is the random walk the right baseline for prices?*

答 A Daily prices are close to a random walk, so “tomorrow equals today” is very hard to beat. Any model has to clear that bar before it means anything.

数据泄漏和对齐问题 · Leakage and misalignment

怎么做 列出 12 个问题逐个量化，包括 **标准化器在测试集上拟合**（泄漏）、**混合模型 60 天错位**、预测长度不一致，以及结论和自己的 CSV 矛盾。

为什么 · 取舍 每个问题都量化，**比只说「有问题」花时间得多**。

追问 Q *What is the danger of fitting the scaler on the test data?*

答 A It leaks information about the future into training, so the model looks better than it would be on truly unseen data.

滚动起点 + DM 检验 · Rolling origins and Diebold-Mariano tests

怎么做 从 **每个交易日** 预测 1、5、20 天收益（208 个起点 × 20 只），测试期放在基础模型发布之后；16 个模型用 **Diebold-Mariano 检验** 和随机游走比，0 个显著更好；波动率倒能预测（误差 −20%，p = 0.048）。

为什么 · 取舍 计算量大很多，**换来不靠一次切分碰运气的结论**。

追问 Q *Why put the test period after the foundation models were released?*

答 A So their training data couldn't include the test period; otherwise they could have seen the answers.

学习视频生成 · Narrated Study Videos

1 **谁用、什么问题** · Who and why

面试要把设计讲出来，光读练不了；自己录屏改一个字就得重录

Explaining a design out loud is the one interview skill you can't practise by reading, and re-recording a screencast for every typo is slow.

2 **它是什么** · What it is

写好一句一行的脚本，一条命令生成带双人配音、中英字幕和小测验的模拟面试视频

So I built a pipeline where I write a walkthrough **one sentence per line**, and one command turns it into a narrated mock interview: two voices, Chinese and English subtitles, and guess-then-reveal quizzes.

3 **怎么运作** · How it works

脚本 → 本地 Kokoro 配音（面试官、候选人两个声音）→ Playwright 截图生成画面 → ffmpeg 合成、统一响度

Each line names who speaks, its Chinese subtitle and the scene to show. Speech comes from **a local Kokoro server**; scenes are rendered as HTML and captured with **Playwright**; **ffmpeg** joins the clips and normalises loudness. Code traces are made by **actually running the code**, so the picture can't drift from what it does.

4 **三个保证** · What you can count on

改一句只重配那一句 · 代码演示和真实运行一致 · 免费（319 句 0 次付费调用）

Three things you can count on. Fix one line and only that line is re-voiced. Code traces always match the real run. And it's free: **0 paid API calls across 319 lines**.

5 **故意放弃的** · Trade-offs

画面是模板生成的，不如手绘灵活；新场景要写一点 Python

Two trade-offs. **Scenes come from templates**, so they're less flexible than drawing by hand, and **a new kind of scene needs a little Python**.

6 **适合谁** · When to use it

要反复修改、长期做系列就值得；只录一次手动录更快

It pays off for a series you'll keep editing. **For a one-off video, recording by hand is faster.**

7 **我做了什么、现在到哪** · My part and status

我一个人做的；已出 5 集（限流、短链、信息流、通知、任务调度，共 34 分钟），代码私有

I built it alone. Five episodes so far: rate limiter, URL shortener, news feed, notifications and job scheduler, 34 minutes in total. The code is private.

8 **一句话收尾** · In one line

脚本是唯一来源，改一句就重生成那一句

In one line: the script is the source of truth, so fixing a sentence only rebuilds that sentence.

关键技术深挖 · Technical deep dive

学习视频生成 · Narrated Study Videos

脚本是唯一来源 · The script is the source of truth

怎么做 每行一句：谁说、中文字幕、显示什么画面。**改一句只重配那一句**。

为什么 · 取舍 写脚本要守格式，**不如随手录灵活**。

追问 Q *How do you avoid re-recording everything for one typo?*

答 A Each sentence is its own audio clip keyed to its line, so changing one line regenerates one clip.

画面是数据，代码演示来自真实运行 · Scenes as data, traces from real runs

怎么做 画面写成数据（表格、流程、代码追踪、数组、链表、树），渲染成 HTML 用 Playwright 截图；**代码追踪是真的跑代码记录每一步并断言结果**，所以画面不会和代码不一致。

为什么 · 取舍 新类型的画面要写一点 Python，**灵活度换一致性**。

追问 Q *Why generate code traces by running the code?*

答 A A hand-drawn trace can be subtly wrong. Recording the real execution and asserting the result means the picture always matches what the code does.

先预览、打分再渲染 · Preview and score before rendering

怎么做 每个不同的画面先截图检查有没有文字溢出；再按评分表检查（小测验、思考停顿、先写代码再运行、一次找 bug、没有超过 15 秒的静止画面），**通过才渲染**；音频统一到 **-16 LUFS**。

为什么 · 取舍 **多一步检查**，但不用整集重渲。

追问 Q *Why normalise to -16 LUFS?*

答 A It's a common loudness target for online video, so every episode plays at the same volume.

LLM 架构卡片 · LLM Arch Cards

1 谁用、什么问题 · Who and why

给 GPU 机器选模型，要先知道注意力结构、KV cache 占多少、真实参数量，最好别先下一个小时权重

Choosing a model for a GPU box means knowing its attention layout, KV-cache cost and real parameter count, ideally before spending an hour downloading weights.

2 它是什么 · What it is

从模型配置读出结构，做成一张卡片，不用下载权重

LLM Arch Cards read an open-weight model's config and show what's inside, **without downloading any weights**.

3 怎么运作 · How it works

在 PyTorch 的 meta 设备上只建形状 → 画出注意力类型、FFN 或 MoE、层结构 → 算 bf16 / int8 / int4 内存 → 和 Hub 上的参数量对账

It instantiates the model on PyTorch's **meta device**, which holds shapes only, so even a 70B model takes seconds and no disk. The card shows the **attention type, FFN or MoE**, the layer schedule, one decoder layer's weight shapes, and **memory for bf16, int8 and int4**. The parameter total is **cross-checked against the Hub's metadata**.

4 三个保证 · What you can count on

快（几秒）· 不占硬盘（0 B 权重）· 数字和官方对得上

Three things you can count on. Seconds per model. **0 bytes of weights downloaded.** And totals that match the Hub, with any difference explained.

5 故意放弃的 · Trade-offs

只看结构和内存，不测速度和效果

The trade-off: **it shows structure and memory, not speed or quality**, so it narrows the choice rather than making it.

6 适合谁 · When to use it

选型第一步先排除放不下的模型，再去跑基准

Use it as the first filter: **rule out models that won't fit, then benchmark the rest**.

7 我做了什么、现在到哪 · My part and status

我一个人做的，MIT 开源，已有 5 张模型卡片

I built it alone under the MIT licence; five model cards are live.

8 一句话收尾 · In one line

不下载权重，先看清模型里面有什么

In one line: see what's inside a model before you download it.

Think in trade-offs ☺

关键技术深挖 · Technical deep dive

LLM 架构卡片 · LLM Arch Cards

meta 设备只建形状 · Instantiate on the meta device

怎么做 在 PyTorch 的 **meta 设备** 上建模型，只有形状没有数据，70B 也只要几秒、0 硬盘。

为什么 · 取舍 **看不到真实权重**，所以只能分析结构和内存。

追问 Q *How can you build a 70B model without downloading it?*

答 A On the meta device, tensors have shapes and types but no storage, so the model's structure exists without any weights.

内存估算 · Memory arithmetic

怎么做 按 **bf16 / int8 / int4** 算权重内存，并画出注意力类型、FFN 或 MoE、层结构和一层的权重形状。

为什么 · 取舍 **不算速度和效果**。

追问 Q *What decides whether a model fits on a GPU?*

答 A Weight memory at the chosen precision plus the KV cache for your context length and batch; the card shows the weight part directly.

和 Hub 对账 · Cross-check against the Hub

怎么做 参数总数和 **Hub 上 safetensors 元数据** 对比，有差异就解释原因。

为什么 · 取舍 多一步核对，**但数字可信**。

追问 Q *Why would the totals differ?*

答 A For example, tied embeddings counted once versus twice, or extra buffers. The card explains each difference instead of hiding it.